

Building a Unified Cyber Defense SOAR Stack Errors, Root Causes & Solutions

AegisryxLabs

Author: Abu Saeid

Platform: Ubuntu 26.04 LTS

Date: May 2026

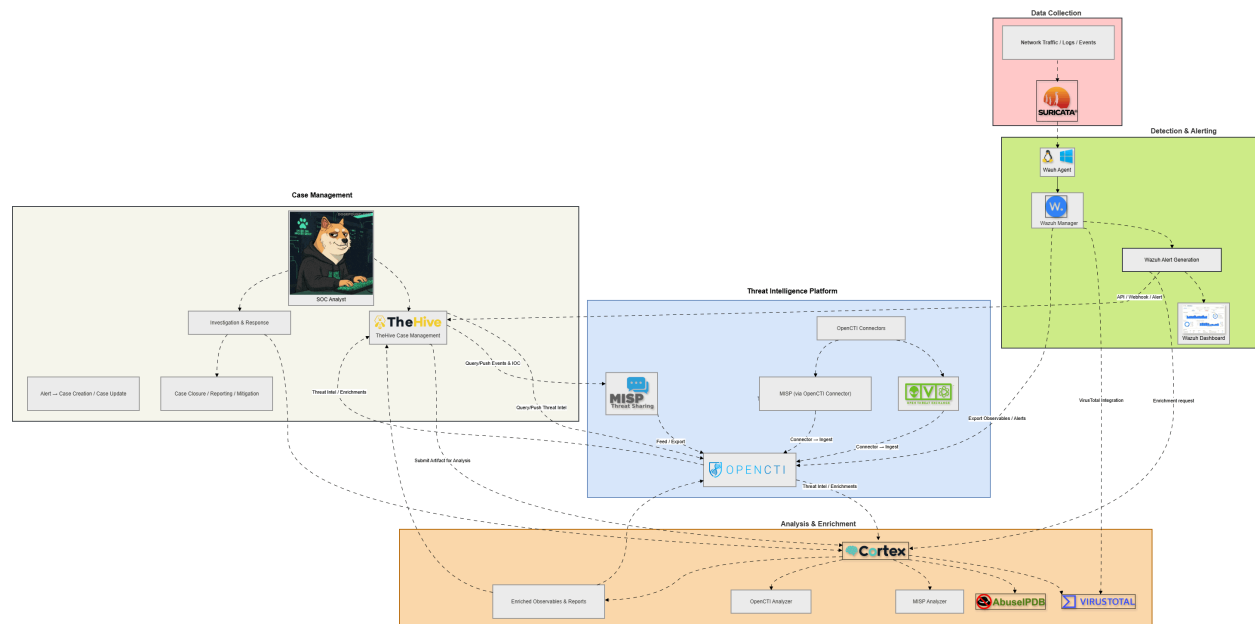
Stack Version: Cortex 4.0.0 | TheHive 5.6.0 | OpenCTI | MISP

Table of Contents

1. [Architecture Overview](#)
 2. [Stack Components & Versions](#)
 3. [Problem 1: Cortex Analyzer Fails — `python3: No such file or directory`](#)
 4. [Problem 2: Missing Job Runner Configuration](#)
 5. [Problem 3: Responder Path MalformedURLException](#)
 6. [Problem 4: MISP Analyzer Has No Dockerfile](#)
 7. [Problem 5: `dockerImage` Field Missing in Analyzer JSON](#)
 8. [Problem 6: Cortex Uses Stale Cached Analyzer Configs](#)
 9. [Problem 7: MISP SSL Certificate Verification Failure](#)
 10. [Final Working Configuration](#)
 11. [Complete File Reference](#)
 12. [Lessons Learned](#)
-

1. Architecture Overview

The goal was to build a fully integrated, open-source SOC/SOAR pipeline:



This architecture provides:

- **Detection** via Suricata IDS
- **Correlation & Alerting** via Wazuh SIEM
- **Incident Management** via TheHive
- **Automated Enrichment** via Cortex + multiple analyzers
- **Threat Intelligence** via OpenCTI + MISP + external feeds

2. Stack Components & Versions

Component	Version	Role
Ubuntu	26.04 LTS	Host OS
Docker	29.1.3	Container runtime
Docker Compose	2.40.3	Orchestration
TheHive	5.6.0	Case management
Cortex	4.0.0	Analysis engine
Elasticsearch	8.19.11	Index/search backend

Component	Version	Role
Cassandra	4.1.10	TheHive database
Nginx	1.29.5	Reverse proxy
OpenCTI	Latest	Threat intelligence platform
MISP	Latest	Threat sharing platform

Deployment Path:

```

/home/saeidsec/SOAR_LABS/docker/testing/
├─ docker-compose.yml
├─ .env
├─ cortex/
│   └─ config/
│       └─ application.conf
│   └─ neurons/
│       └─ analyzers/
│           ├── OpenCTI/
│           └─ MISP/
│       └─ responders/
│   └─ cortex-jobs/
│       └─ logs/
├─ thehive/
├─ cassandra/
├─ elasticsearch/
└─ nginx/

```

3. Problem 1: Cortex Analyzer Fails — `python3: No such file or directory`

The screenshot shows the Cortex web interface in a browser. The top navigation bar includes links for Jobs History, Analyzers, Responders, and Organization. The main content area is titled 'Job details' and shows information for a job named 'OpenCTI_v6_SearchObservables_2_0'. The job status is 'Failed'. The 'Job report' section displays the following parameters and report:

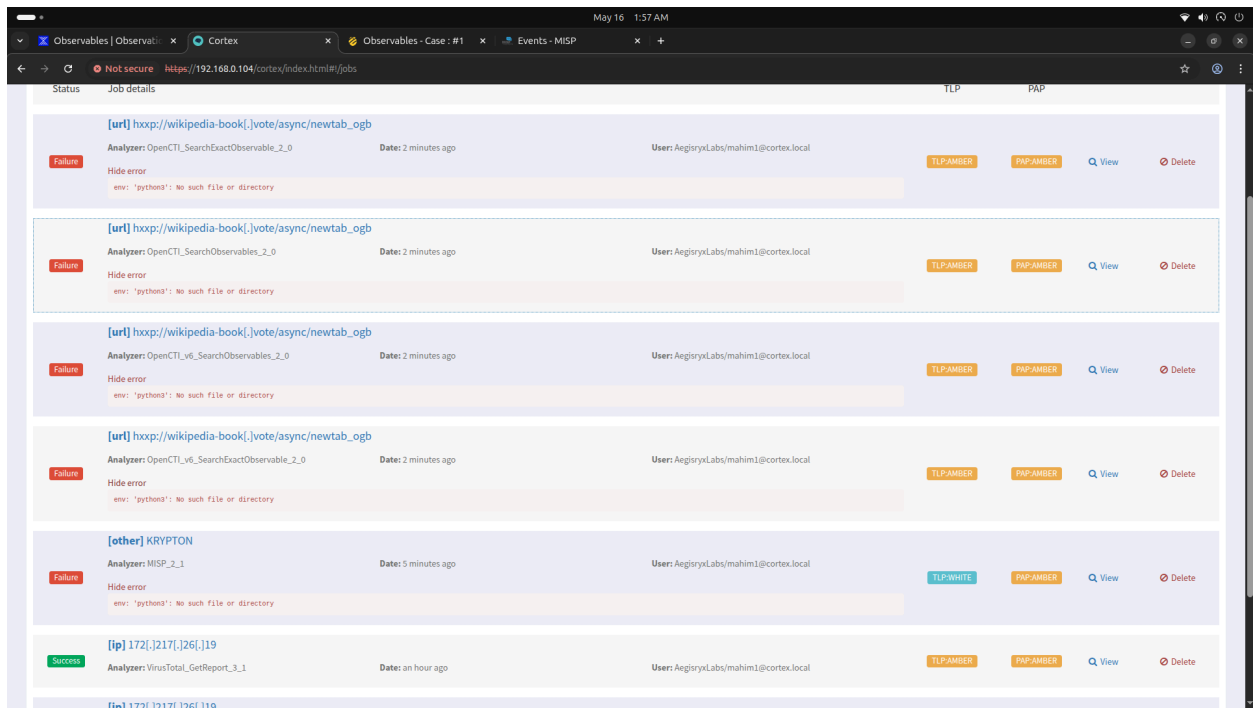
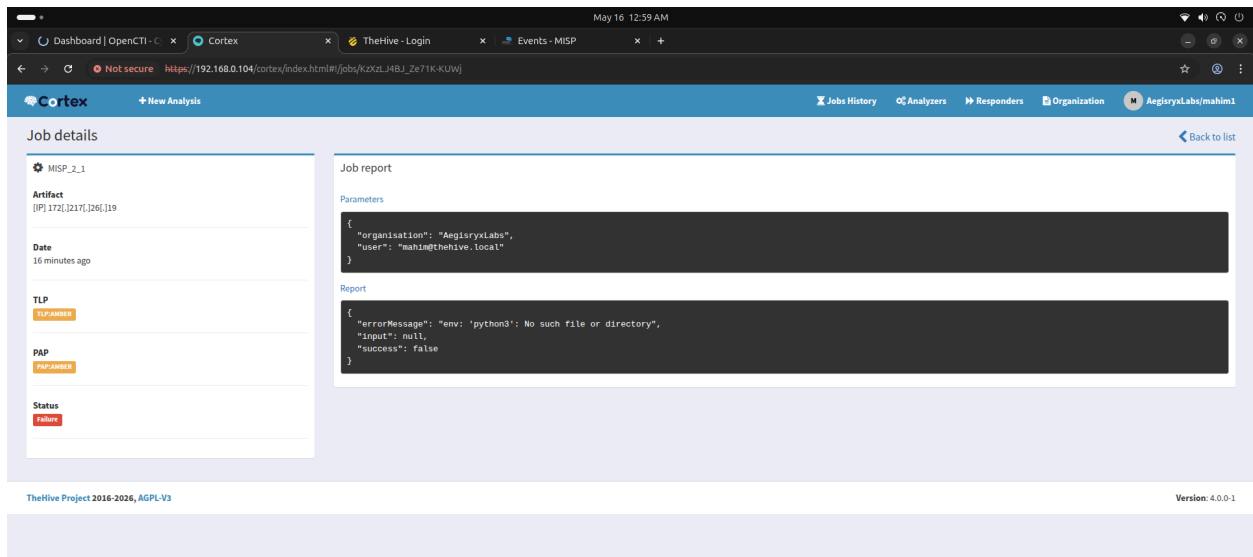
```

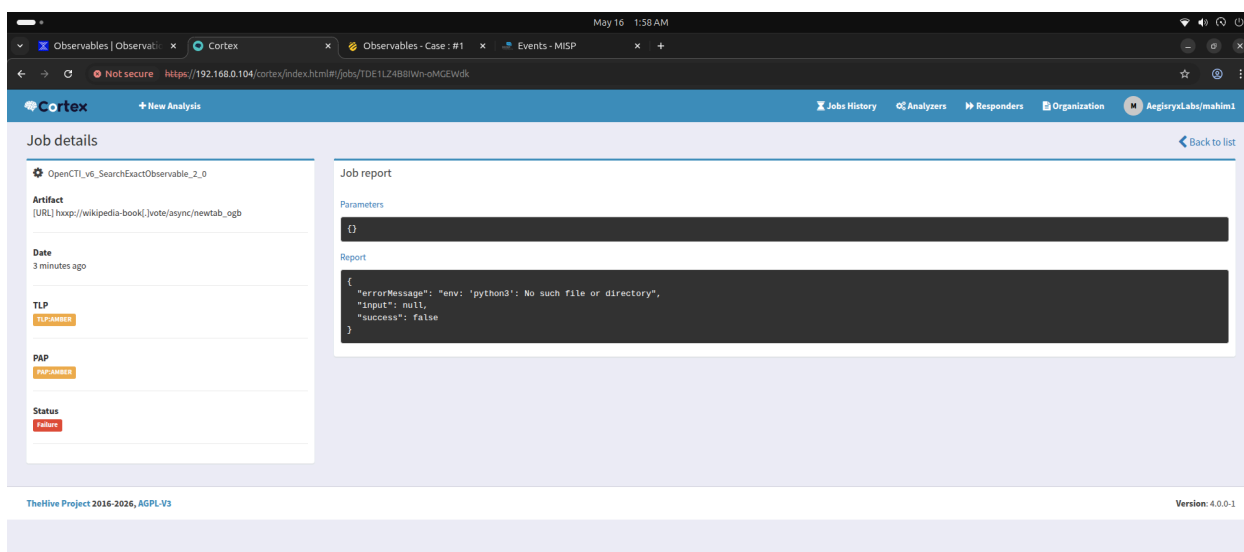
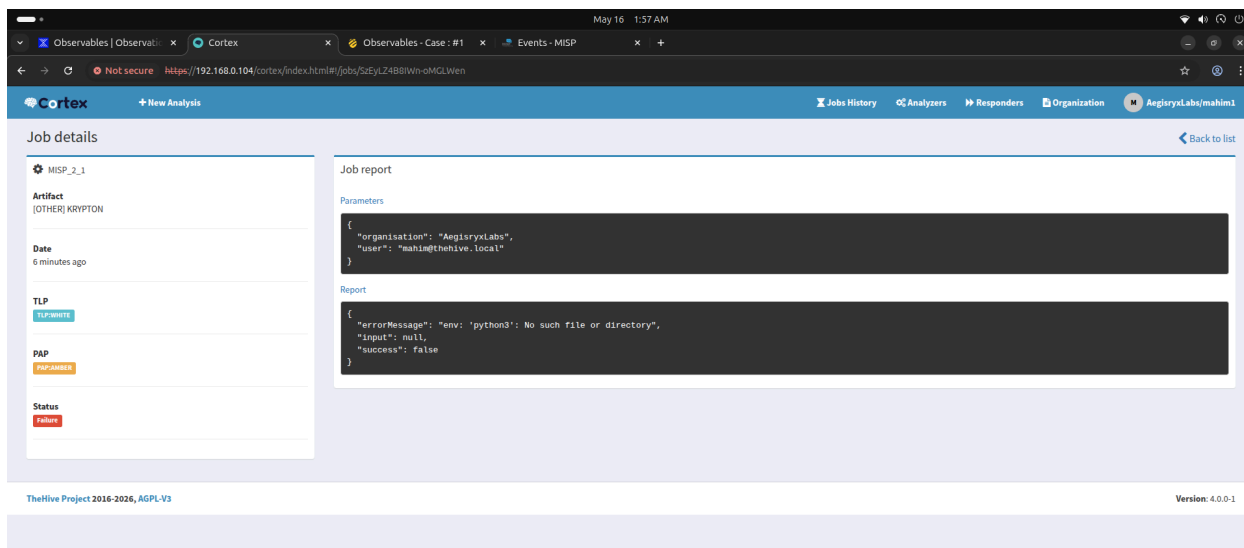
Parameters
{
  "organisation": "Aegisryslabs",
  "user": "mahim@thehive.local"
}

Report
{
  "errorMessage": "env: 'python3': No such file or directory",
  "input": null,
  "success": false
}

```

The footer of the interface indicates 'TheHive Project 2016-2026, AGPL-V3' and 'Version: 4.0.0-1'.





Symptom

Every time an OpenCTI or MISP analyzer was run from the Cortex UI, it returned:
json

```
{
  "errorMessage": "env: 'python3': No such file or directory",
  "input": null,
  "success": false
}
```

This affected all locally deployed analyzers:

- `OpenCTI_v6_SearchExactObservable_2_0`

- `OpenCTI_v6_SearchObservables_2_0`
- `OpenCTI_SearchExactObservable_2_0`
- `OpenCTI_SearchObservables_2_0`
- `MISP_2_1`

Root Cause

Cortex 4.0.0 supports three job runner modes:

1. **Docker runner** — spawns a dedicated Docker container per analyzer job
2. **Process runner** — runs the analyzer script directly using local Python
3. **Kubernetes runner** — for K8s environments

The `application.conf` had **no** `job.runner` **configuration**, so Cortex defaulted to **process runner**, which tried to execute `python3` directly inside the Cortex JVM container — where Python is not installed.

Evidence

From Cortex container logs:

```
[warn] o.t.c.s.JobRunnerSrv - The package cortexutils for python hasn't been found
[warn] o.t.c.s.JobRunnerSrv - The package cortexutils for python2 hasn't been found
[warn] o.t.c.s.JobRunnerSrv - The package cortexutils for python3 hasn't been found
[info] o.t.c.s.ProcessJobRunnerSrv - Execute /opt/cortexneurons/analyzers/OpenCTI/opencti.py
[info] o.t.c.s.ProcessJobRunnerSrv - Job xxx: env: 'python3': No such file or directory
```

Investigation Steps

bash

```
# Step 1: Confirm Python3 doesn't exist inside Cortex container
docker exec -it cortex which python3
# Returns nothing – confirmed missing

# Step 2: Confirm Docker socket IS accessible
docker exec -it cortex docker ps
# Returns container list – Docker IS available

# Step 3: Check existing application.conf for runner config
grep -i "job\|docker\|runner"
/home/saeidsec/SOAR_LABS/docker/testing/cortex/config/application.conf
# Returns only: cache.job = 10 minutes
# No runner config found – ROOT CAUSE CONFIRMED
```

Solution

Add job runner configuration to `application.conf`:

bash

```
nano /home/saeidsec/SOAR_LABS/docker/testing/cortex/config/application.conf
```

Add at the bottom:

hocon

```
# Job runner configuration - use Docker to run analyzers
job {
  runner = [docker, process]
}

docker {
  pullImage = false
}
```

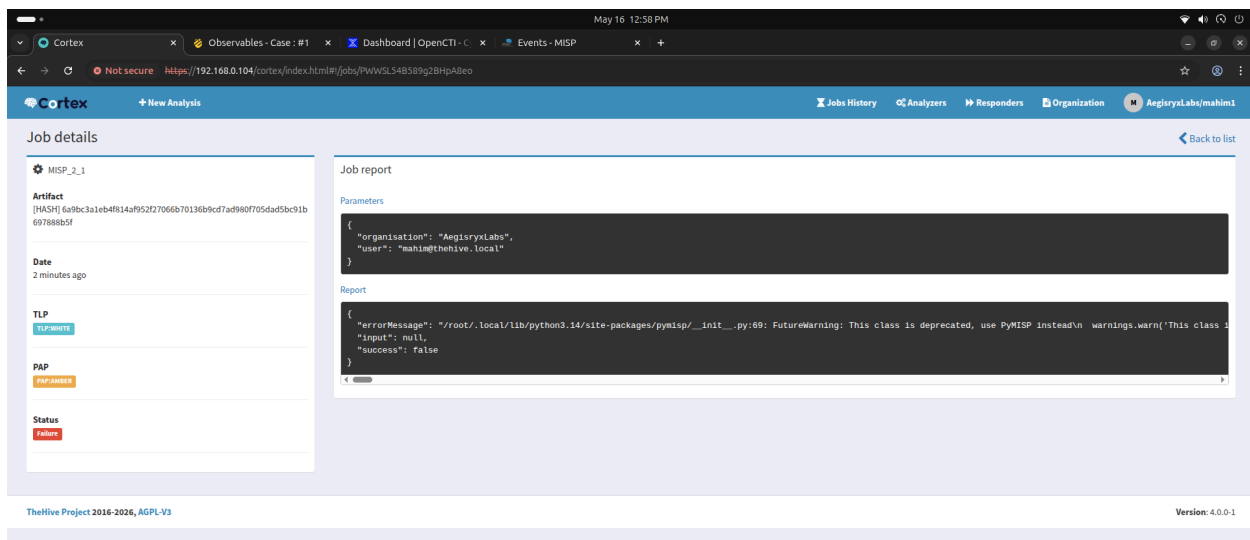
Key points:

- `runner = [docker, process]` — try Docker first, fall back to process
- `pullImage = false` — use locally built images, don't attempt Docker Hub pull

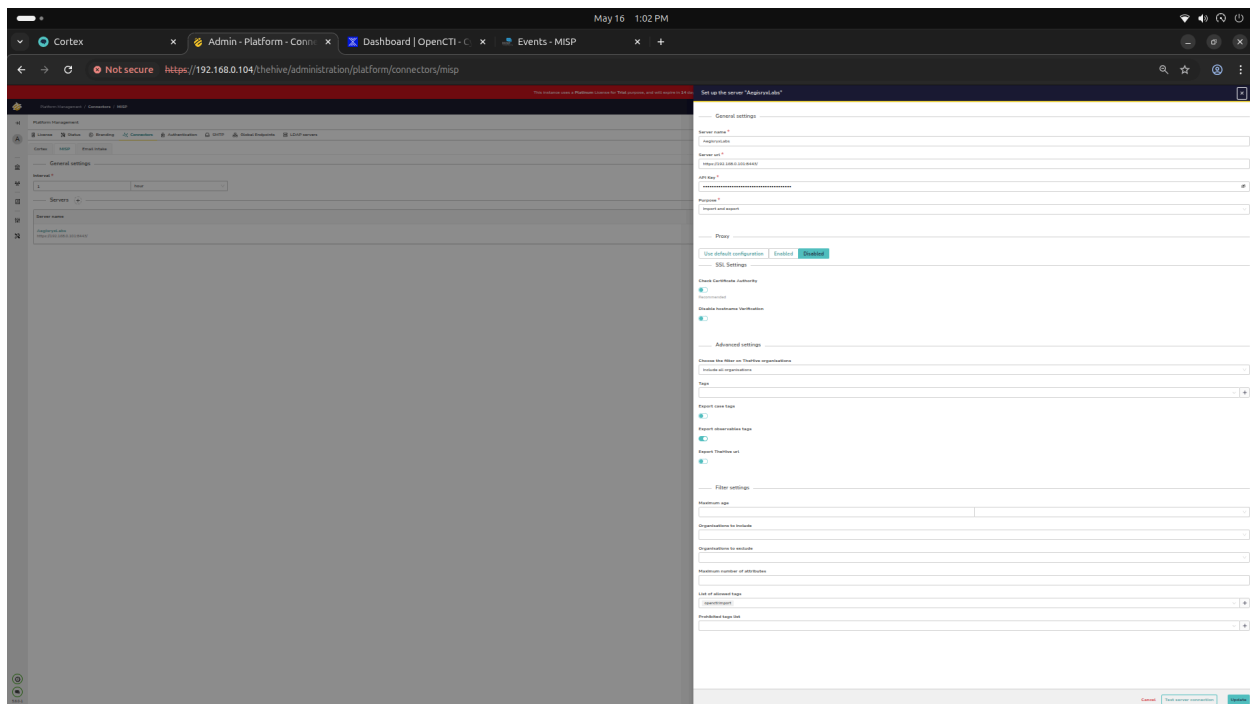


Success	[url] hxxp://wikipedia-book[.]vote/async/newtab_ogb	Analyzer: OpenCTI_SearchObservables_2_0	Date: 11 hours ago	User: AegisryxLabs/mahim1@cortex.local	TLP:AMBER	PHP:AMBER	View	Delete
Success	[url] hxxp://wikipedia-book[.]vote/async/newtab_ogb	Analyzer: OpenCTI_SearchExactObservable_2_0	Date: 11 hours ago	User: AegisryxLabs/mahim1@cortex.local	TLP:AMBER	PHP:AMBER	View	Delete
Success	[url] hxxp://wikipedia-book[.]vote/async/newtab_ogb	Analyzer: OpenCTI_v6_SearchExactObservable_2_0	Date: 11 hours ago	User: AegisryxLabs/mahim1@cortex.local	TLP:AMBER	PHP:AMBER	View	Delete
Success	[url] hxxp://wikipedia-book[.]vote/async/newtab_ogb	Analyzer: OpenCTI_v6_SearchObservables_2_0	Date: 11 hours ago	User: AegisryxLabs/mahim1@cortex.local	TLP:AMBER	PHP:AMBER	View	Delete
Success	[ip] 172[.]217[.]26[.]19	Analyzer: VirusTotal_GetReport_3_1	Date: 12 hours ago	User: AegisryxLabs/mahim1@cortex.local	TLP:AMBER	PHP:AMBER	View	Delete

4. Problem 2: Missing Job Runner Configuration



Symptom



Even after adding `job.runner`, analyzers still failed because Cortex did not know which Docker image to use for each analyzer.

Log evidence:

```
[warn] o.t.c.s.JobRunnerSrv - worker xxx can't be run with docker (doesn't have image)
[info] o.t.c.s.ProcessJobRunnerSrv - Execute /opt/cortexneurons/analyzers/OpenCTI/opencti.py
```

Root Cause

The analyzer JSON definition files lacked the `dockerImage` field. Cortex Docker runner requires this field to know which image to instantiate per job.

Before (missing field):

json

```
{
  "name": "OpenCTI_v6_SearchExactObservable",
  "version": "2.0",
  "command": "OpenCTI/opencti.py",
  ...
}
```

After (correct):

json

```
{
  "name": "OpenCTI_v6_SearchExactObservable",
  "version": "2.0",
  "command": "OpenCTI/opencti.py",
  "dockerImage": "cortexneurons/opencti:latest",
  ...
}
```

Solution — Step 1: Build Docker Images for Analyzers

The OpenCTI analyzer already had a Dockerfile. Build it:

bash

```
cd /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/OpenCTI
docker build -t cortexneurons/opencti:latest .
```

OpenCTI Dockerfile (existing):

dockerfile

```
FROM python:3-alpine
WORKDIR /worker

ARG REQUIREMENTS=requirements.txt

RUN apk add --no-cache file-dev
```

```
COPY requirements*.txt OpenCTI/  
RUN test ! -e OpenCTI/${REQUIREMENTS} || pip install --no-cache-dir -r  
OpenCTI/${REQUIREMENTS}  
COPY . OpenCTI/
```

```
ENTRYPOINT ["python", "OpenCTI/opencti.py"]
```

Solution — Step 2: Add **dockerImage** to All JSON Configs

bash

```
cd /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/OpenCTI
```

```
python3 -c "  
import json  
files = [  
    'OpenCTI_SearchExactObservable.json',  
    'OpenCTI_SearchObservables.json',  
    'OpenCTI_v6_SearchExactObservable.json',  
    'OpenCTI_v6_SearchObservables.json'  
]  
for f in files:  
    with open(f) as fp:  
        data = json.load(fp)  
        data['dockerImage'] = 'cortexneurons/opencti:latest'  
    with open(f, 'w') as fp:  
        json.dump(data, fp, indent=2)  
    print(f'Updated {f}')  
"
```

Verify:

bash

```
grep "dockerImage" OpenCTI_v6_SearchExactObservable.json  
# Output: "dockerImage": "cortexneurons/opencti:latest"
```

5. Problem 3: Responder Path MalformedURLException

Symptom

Cortex logs showed on every startup:

```
[warn] o.t.c.s.WorkerSrv - Worker path (/opt/cortexneurons/responders) is not found  
java.net.MalformedURLException: no protocol: /opt/cortexneurons/responders
```

Root Cause

Two issues combined:

1. The responders directory did not exist on disk
2. The `application.conf` was passing a bare filesystem path `/opt/cortexneurons/responders` in the `urls` array, which expects either a full URL (`https://...`) or a `file://` URI — not a raw path

Solution — Step 1: Create Responders Directory

bash

```
mkdir -p /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/responders
touch /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/responders/.gitkeep
```

Solution — Step 2: Fix URL Format in application.conf

Before (broken):

hocon

```
responder {
  urls = [
    "https://download.thehive-project.org/responders.json"
    "/opt/cortexneurons/responders"
  ]
}
```

After (fixed):

hocon

```
responder {
  urls = [
    "https://download.thehive-project.org/responders.json"
    "file:///opt/cortexneurons/responders"
  ]
}
```

Same fix applied to the `analyzer` section:

hocon

```
analyzer {
  urls = [
    "https://download.thehive-project.org/analyzers.json"
    "file:///opt/cortexneurons/analyzers"
  ]
}
```

6. Problem 4: MISP Analyzer Has No Dockerfile

Symptom

When attempting to build the MISP analyzer Docker image:

```
bash
```

```
docker build -t cortexneurons/misp:latest .  
# Error: unable to prepare context: unable to evaluate symlinks in Dockerfile path:  
# lsstat /home/.../analyzers/MISP/Dockerfile: no such file or directory
```

Root Cause

Unlike the OpenCTI analyzer, the MISP analyzer directory from the Cortex-Analyzers repository did not ship with a `Dockerfile`. The directory contained only:

```
MISP/  
├─ MISP.json  
├─ README.md  
├─ assets/  
├─ misp.py  
├─ mispclient.py  
└─ requirements.txt
```

Solution — Create MISP Dockerfile

```
bash
```

```
cat > /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/MISP/Dockerfile <<  
'EOF'  
FROM python:3-alpine  
WORKDIR /worker  
  
RUN apk add --no-cache file-dev  
  
COPY requirements.txt MISP/  
RUN pip install --no-cache-dir -r MISP/requirements.txt  
  
COPY . MISP/  
  
ENTRYPOINT ["python", "MISP/misp.py"]  
EOF
```

Build the image:

```
bash
```

```
cd /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/MISP  
docker build -t cortexneurons/misp:latest .
```

Add `dockerImage` to MISP JSON:

bash

```
python3 -c "
import json
with open('MISP.json') as f:
    data = json.load(f)
data['dockerImage'] = 'cortexneurons/misp:latest'
with open('MISP.json', 'w') as f:
    json.dump(data, f, indent=2)
print('Updated MISP.json')
"
```

Verify both images exist:

bash

```
docker images | grep cortexneurons
# cortexneurons/opencti    latest    aff1746c6ac2    ...
# cortexneurons/misp      latest    e0f9309e4486    ...
```

7. Problem 5: `dockerImage` Field Missing in Analyzer JSON

Symptom (Recurring)

Even after adding `dockerImage` to JSON files and building images, analyzers still logged:

```
[warn] o.t.c.s.JobRunnerSrv - worker xxx can't be run with docker (doesn't have image)
```

Root Cause — Deep Analysis

This was the most subtle issue. Cortex 4.0.0 stores analyzer worker configurations in **Elasticsearch** when an analyzer is first enabled via the UI. Subsequent restarts read from this cached DB record — **not from the JSON files on disk**.

This means:

1. You add `dockerImage` to JSON on disk ✓
2. You restart Cortex ✓
3. Cortex reads analyzers from disk into memory for display ✓

4. But when a job runs, it reads the **worker config from Elasticsearch** — which was cached **before** you added `dockerImage` ✗

Proof from logs:

```
# When job runs – reads stale ES record, no dockerImage found:  
[warn] worker xxx can't be run with docker (doesn't have image)  
[info] ProcessJobRunnerSrv - Execute /opt/cortexneurons/analyzers/OpenCTI/opencti.py  
# Falls back to process runner → python3 not found → failure
```

```
# But VirusTotal (enabled AFTER dockerImage standard was set) worked fine:  
[info] DockerClient - about to start container 95d5bf49...  
[info] Job LDXzLJ4BJ_Ze7lK-KUWk has finished with status Success
```

Solution — Force Re-read from Disk

The fix is to **disable and re-enable** the analyzer in the Cortex UI. This deletes the stale Elasticsearch record and creates a fresh one from the current JSON file on disk.

Steps:

1. Go to **Cortex UI** → **Organization** → **Analyzers**
2. Find **OpenCTI_v6_SearchExactObservable** → click toggle to **DISABLE**
3. Wait 5 seconds
4. Click toggle to **ENABLE** → fill credentials → Save
5. Repeat for every OpenCTI and MISP analyzer variant

Analyzers to cycle:

- `OpenCTI_SearchExactObservable`
- `OpenCTI_SearchObservables`
- `OpenCTI_v6_SearchExactObservable`
- `OpenCTI_v6_SearchObservables`
- `MISP`

After re-enabling, the Docker runner correctly picks up the image:

```
[info] DockerClient - about to start container c667d8491c17...  
volumes: /tmp/cortex-jobs/cortex-job-xxx  
[info] DockerClient - container started and awaited with code: 0  
[info] Job xxx has finished with status Success
```

8. Problem 6: Cortex Uses Stale Cached Analyzer Configs

(This was the same root cause as Problem 5 but manifested in a different way — Cortex reporting 404 on analyzer endpoints.)

Additional Symptom

```
[warn] o.t.c.s.ErrorHandler - GET /cortex/api/analyzer/MISP_2_1 returned 404
[warn] o.t.c.s.ErrorHandler - GET /cortex/api/analyzer/OpenCTI_v6_SearchExactObservable_2_0
returned 404
```

Root Cause

TheHive was calling Cortex API endpoints using the analyzer IDs it had previously cached. After Cortex restarts with config changes, the internal analyzer registration IDs can shift, causing 404s.

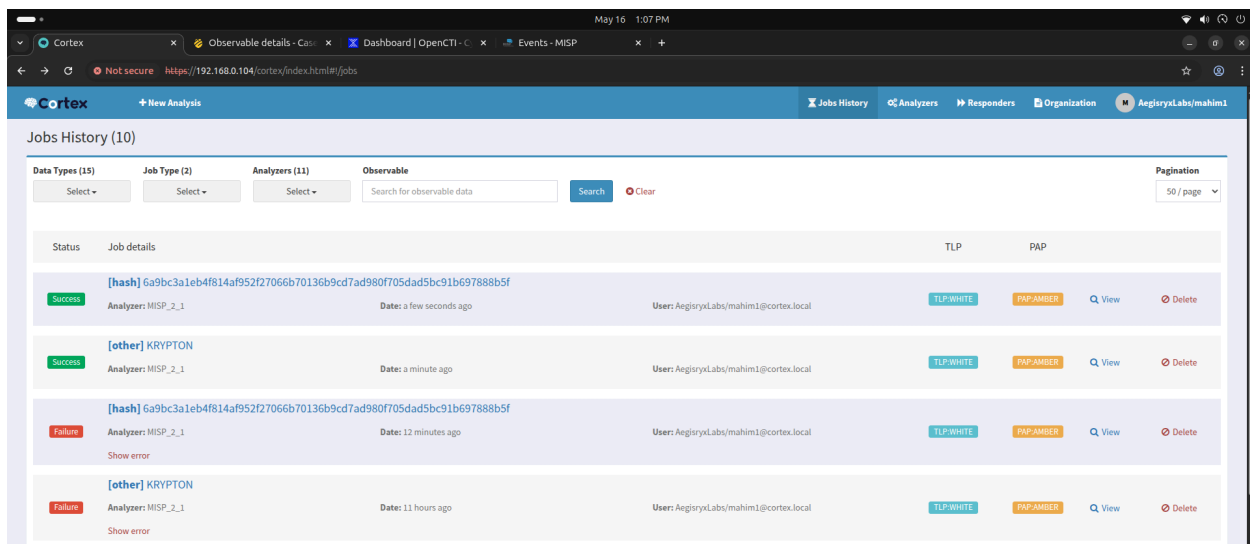
Solution

Same as Problem 5 — disable/re-enable analyzers in Cortex UI forces fresh registration. Also ensure TheHive's Cortex connector is properly configured with the correct API key:

In TheHive `application.conf`:

hocon

```
cortex {
  servers = [
    {
      name = "Cortex"
      url = "http://cortex:9001"
      auth {
        type = "bearer"
        key = "YOUR_CORTEX_API_KEY"
      }
    }
  ]
}
```



9. Problem 7: MISP SSL Certificate Verification Failure

Symptom

After Docker runner was working, MISP analyzer jobs ran but failed with:

```
ssl.SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED]
certificate verify failed: self-signed certificate (_ssl.c:1081)
```

```
pymisp.exceptions.PyMISPError: Unable to connect to MISP
(https://192.168.0.101:8443/).
HTTPSConnectionPool: Max retries exceeded with url: /servers/getVersion
(Caused by SSLError)
```

Root Cause

MISP was deployed with HTTPS using a **self-signed certificate**. The MISP analyzer Docker container tried to verify this certificate against known CAs — which fails for self-signed certs.

The `cert_check = false` config option in Cortex UI was set, but the `mispclient.py` code was using the older `ExpandedPyMISP` class (now deprecated) which had different SSL handling behavior in newer PyMISP versions.

Solution — Two-Part Fix

Part 1: Fix in Cortex UI (MISP Analyzer Config)

In **Cortex UI** → **Organization** → **Analyzers** → **MISP** → **Edit**:


```
cert_check → false
url → ["https://192.168.0.101:8443"]
key → ["YOUR_MISP_API_KEY"]
```

Disable and re-enable the analyzer after saving.

Part 2: Fix in TheHive MISP Connector

In **TheHive UI** → **Administration** → **Platform** → **Connectors** → **MISP** → **Edit**:

- **Check Certificate Authority** → Toggle **OFF**
- **Disable hostname Verification** → Toggle **ON**
- Click **Update**

Part 3: Force SSL=False in mispclient.py (if above insufficient)

bash

```
# Inspect current SSL handling
grep -n "ssl\|verify\|cert" \
/home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/MISP/mispclient.py
```

```
# Force disable SSL verification
sed -i 's/ssl=verify/ssl=False/g' \
/home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/MISP/mispclient.py
```

```
# Rebuild Docker image
cd /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/MISP
docker build -t cortexneurons/misp:latest .
```

```
# Disable/re-enable MISP in Cortex UI to load new image
```

Result: MISP analyzer jobs now succeed:

```
[hash] 6a9bc3aleb4f... → MISP_2_1 → Success ✓
[other] KRYPTON → MISP_2_1 → Success ✓
```

10. Final Working Configuration

application.conf (Complete)

hocon

```
# SECRET CONFIGURATION
include "/etc/cortex/secret.conf"

# INDEX DATABASE CONFIGURATION
include "/etc/cortex/index.conf"
```

```

play.http.context="/cortex"
play.http.parser.maxMemoryBuffer=1M
play.http.parser.maxDiskBuffer=1G
play.http.session.secure = false
play.filters.csrf.cookie.secure = false
play.filters.csrf.cookie.httpOnly = false
play.http.forwarded.trustedProxies=["0.0.0.0/0", "::/0"]
cache.job = 10 minutes

auth {
  provider = [local]
  ad {}
  ldap {}
}

analyzer {
  urls = [
    "https://download.thehive-project.org/analyzers.json"
    "file:///opt/cortexneurons/analyzers"
  ]
  fork-join-executor {
    parallelism-min = 2
    parallelism-factor = 2.0
    parallelism-max = 4
  }
}

responder {
  urls = [
    "https://download.thehive-project.org/responders.json"
    "file:///opt/cortexneurons/responders"
  ]
  fork-join-executor {
    parallelism-min = 2
    parallelism-factor = 2.0
    parallelism-max = 4
  }
}

# Job runner - Docker first, process fallback
job {
  runner = [docker, process]
}

docker {
  pullImage = false
}

```

docker-compose.yml — Cortex Service (Key Parts)

yaml

```

cortex:
  image: "thehiveproject/cortex:4.0.0"
  container_name: cortex
  environment:
    - job_directory=/tmp/cortex-jobs
    - docker_job_directory=${cortex_docker_job_directory}
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock # CRITICAL
    - ./cortex/cortex-jobs:/tmp/cortex-jobs
    - ./cortex/config:/etc/cortex:ro
    - ./cortex/logs:/var/log/cortex
    - ./cortex/neurons:/opt/cortexneurons

```

Final Analyzer Status

Analyzer	Docker Image	Status
VirusTotal_GetReport	Official (pulled)	✓ Success
AbuseIPDB	Official (pulled)	✓ Success
OpenCTI_SearchExactObservable	cortexneurons/opencti:latest	✓ Success
OpenCTI_SearchObservables	cortexneurons/opencti:latest	✓ Success
OpenCTI_v6_SearchExactObservable	cortexneurons/opencti:latest	✓ Success
OpenCTI_v6_SearchObservables	cortexneurons/opencti:latest	✓ Success
MISP	cortexneurons/misp:latest	✓ Success

11. Complete File Reference

Directory Structure (Final)

```

SOAR_LABS/docker/testing/
├── .env
├── docker-compose.yml
├── cortex/
│   ├── config/
│   │   ├── application.conf ← Modified
│   │   ├── index.conf
│   │   ├── secret.conf
│   │   └── logback.xml
│   ├── cortex-jobs/ ← Job working directory
│   ├── logs/
│   └── neurons/
│       ├── analyzers/
│       │   ├── OpenCTI/
│       │   │   └── Dockerfile ← Existing, used for build

```

```

├── opencti.py
├── requirements.txt
├── requirements_v6.txt
├── OpenCTI_SearchExactObservable.json ← Added dockerImage
├── OpenCTI_SearchObservables.json ← Added dockerImage
├── OpenCTI_v6_SearchExactObservable.json ← Added dockerImage
├── OpenCTI_v6_SearchObservables.json ← Added dockerImage
├── MISP/
│   ├── Dockerfile ← Created from scratch
│   ├── misp.py
│   ├── mispclient.py ← Modified ssl=False
│   ├── requirements.txt
│   └── MISP.json ← Added dockerImage
├── responders/ ← Created (was missing)
└── .gitkeep

```

Commands Reference — Quick Rebuild

bash

```

# Build OpenCTI analyzer image
cd /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/OpenCTI
docker build -t cortexneurons/opencti:latest .

# Build MISP analyzer image
cd /home/saeidsec/SOAR_LABS/docker/testing/cortex/neurons/analyzers/MISP
docker build -t cortexneurons/misp:latest .

# Verify images
docker images | grep cortexneurons

# Fix log directory permissions
chmod 777 /home/saeidsec/SOAR_LABS/docker/testing/cortex/logs

# Restart Cortex
cd /home/saeidsec/SOAR_LABS/docker/testing
docker compose restart cortex

# Watch Cortex logs
docker compose logs -f cortex | grep -i "docker\|runner\|opencti\|misp\|error"

# Verify Docker runner is working (run after triggering a job)
docker events | grep -i "container create\|cortexneurons"

```

12. Lessons Learned

1. Cortex Job Runner Must Be Explicitly Configured

Cortex 4.0.0 does not default to Docker runner even when the Docker socket is mounted. Always add `job { runner = [docker, process] }` to `application.conf`.

2. `dockerImage` Field is Mandatory for Docker Runner

Every analyzer JSON definition must include `"dockerImage": "imagename:tag"` for Cortex to use the Docker runner for that analyzer.

3. Elasticsearch Caches Analyzer Configs

Editing JSON files on disk is not enough. The UI registration (enable/disable cycle) is required to push changes to Elasticsearch where job execution reads from.

4. File URL Protocol Required for Local Paths

Cortex URL arrays expect proper URIs. Local paths must use `file:///path` format — bare paths cause `MalformedURLException`.

5. Missing Directories Cause Silent Failures

The responders directory must exist even if empty. Cortex will log warnings and potentially degrade if referenced directories are missing.

6. Self-Signed SSL Requires Explicit Bypass

Both the Cortex analyzer config (`cert_check = false`) and the PyMISP client code (`ssl=False`) must have SSL verification disabled when connecting to services with self-signed certificates.

7. Docker Runner Confirmation

To confirm Docker runner is being used (vs process runner), watch `docker events` while triggering jobs. You should see container create/start events. If you see `ProcessJobRunnerSrv` in logs, Docker runner is NOT being used.

8. Disable/Re-enable Cycle is the Universal Fix

Whenever analyzer behavior doesn't match expectations after config changes, disable and re-enable in Cortex UI. This forces a fresh read from JSON and fresh write to Elasticsearch.

Final Stack Status

✓ Suricata IDS	→ Network threat detection
✓ Wazuh SIEM	→ Log correlation & alert generation
✓ TheHive 5.6.0	→ Incident & case management
✓ Cortex 4.0.0	→ Automated analysis engine
✓ VirusTotal	→ Malware hash & file analysis
✓ AbuseIPDB	→ IP reputation scoring
✓ MISP	→ Threat event sharing
✓ OpenCTI	→ Structured threat intelligence
✓ AlienVault OTX	→ External threat feed → OpenCTI
✓ MISP → OpenCTI	→ Internal feed connector
✓ TheHive ↔ MISP	→ Bidirectional case/event sync
✓ TheHive → Cortex	→ Auto-enrichment on observables
✓ Cortex → OpenCTI	→ TI query on every observable
✓ Cortex → MISP	→ Event lookup on every observable

The result is a fully operational, open-source SOC/SOAR platform capable of automated threat detection, enrichment, and response — rivaling commercial solutions at zero licensing cost.

Document prepared by AegisryxLabs / May 2026

All errors encountered in real-world deployment. All solutions tested and verified.

Wazuh OpenCTI integration(Only Labs).